

Kalshiman

A bot that watches the markets all night, so you don't have to.

The full breakdown - for investors, classes, and the curious - 2026-08-20 04:36 UTC

Inside: the edge, the three layers that produce it, the exact rules, the measured receipts, the economics at every scale from one college kid to a thousand bots, the fleet vision - and the honest risks.

A note before the numbers

Every fifteen minutes, Kalshi runs a market on gold, silver, or crypto: up or down in the next quarter hour. Most of the time, the market already knows the answer before the round is over - a side priced at 92 cents settles that way about 97% of the time. That isn't a secret. The secret is having the patience to wait for the 92-cent moment, the nerve to buy it, and the self-control to let it finish instead of flinching at the first dip.

This document is a cheatsheet. It explains, in plain language and with the actual code's rules, how to build a bot that - done correctly - makes enough that a person who runs one doesn't have to work the way they work now. That goal is extreme, and we say so plainly. It depends on progress, on adoption, and on markets that cooperate. But the direction is real, and the receipts are in this document.

The first version of this bot lost money and taught us everything. The rebuilt version ran a perfect hour on paper - eight trades, eight wins, zero drawdown - and then kept going. The real account is untouched. Everything in this document is paper, and it stays that way until the record earns it.

Here is the honest version of the dream: a family of four can live on about \$1,800 a month in the US. One bot, deployed correctly, projects more than that. A college student can run one from a dorm and cover food and fun. A group of friends can run a small fleet for hobby money. A determined individual could run a hundred. A network of a thousand, trading the current strategy at \$20 trades, projects millions a month - before liquidity and reality say their piece. Those numbers are in chapter seven, with the caveats right next to them.

1. The 92-cent question

Walk up to a gold market fifteen minutes before it closes. The price is 92 cents for “up.” Ask a stranger what happens and they'll shrug. Ask the market itself, and it has already told you: the buyers have decided, and almost all of the time - 97% of the time on our measured record - that side is right.

So where's the catch? The catch is that 97% isn't 100%. The other 3% of the time, the 92-cent side collapses, and a loss costs you 92 cents - almost eleven times what a win pays. That's the whole game in one paragraph: win small, often. Lose small, rarely. Never let the two get out of balance.

The measured convergence, from 105,000 finished rounds in the dataset (the favored side settles as predicted):

Market	Held rounds	Resolved as favored
Silver hourly	8,225	98.5%
Gold 15m	7,563	97.1%
Crypto 15m (BTC/ETH/SOL/XRP/DOGE/ZEC)	~24,000	98-99.4%
Oil 15m (WTI)	1,580	99.2%

The old bot got the first half of the game right and the second half catastrophically wrong. It won 79% of the time and lost money anyway, because its losses averaged five times its wins. We also learned that trading fails in exactly two directions. Too hot: trust every hunch, chase every wiggle, get stopped out on noise. Too cold: wait for certainty that never comes, or let a formula stop you out of winners because it can't tell a dip from a disaster.

The fix turned out to be a balance, not a formula: three layers, each one catching the other two's failure mode. The statistician, the observer, and the disciplined hand. Chapter four is about them.

2. What we learned the hard way

Before the rebuild, the bot ran on real money for a few weeks. The record is not pretty, and we're not going to pretend it is. It's the reason everything after it exists.

What we measured	Value
Settled trades	291
Win rate	78.7%
Net P&L	-\$77.85
Worst drawdown	-\$91.43
Average win / average loss	+12.3c / -48.5c
Median loss	-100.7c (a full reversal)

The math is one line: with 12-cent wins and 48-cent losses, break-even needs a 134% win rate. The bot was winning at 79% and losing anyway. Three specific failures explain almost all of it, and each one maps to a specific fix in the code:

- **Stops that didn't stop.** The fast stop monitor only watched live trades; paper trades rode the 5-second tick, so a 15-cent stop booked losses of 17 to 65 cents. Fix: the 1.5-second stop loop now covers paper and live.

- **Stops that ate winners.** Nine of fifteen stopped trades recovered and won at the bell. A dip on a high-confidence pick is usually noise. Fix: the three-minute recovery grace and the final-minute hold.

- **No journal.** The bot never asked what happened after a trade. Fix: every close now gets a written post-mortem - result, what holding would have made, and a verdict that feeds retraining.

Gold was the leak in miniature: 110 trades, a 75% win rate, and an average loss of \$5.46. The single-trade disasters were exact-spot markets, combos, and one hourly oil trade. All of them are banned now.

3. The rebuild: data before opinions

We stopped guessing and started measuring. We pulled every finished round we could find - 105,000 of them, across 16 series - and laid them out the way a trader sees them: what the market looked like before the entry, how it moved afterward, and where it landed.

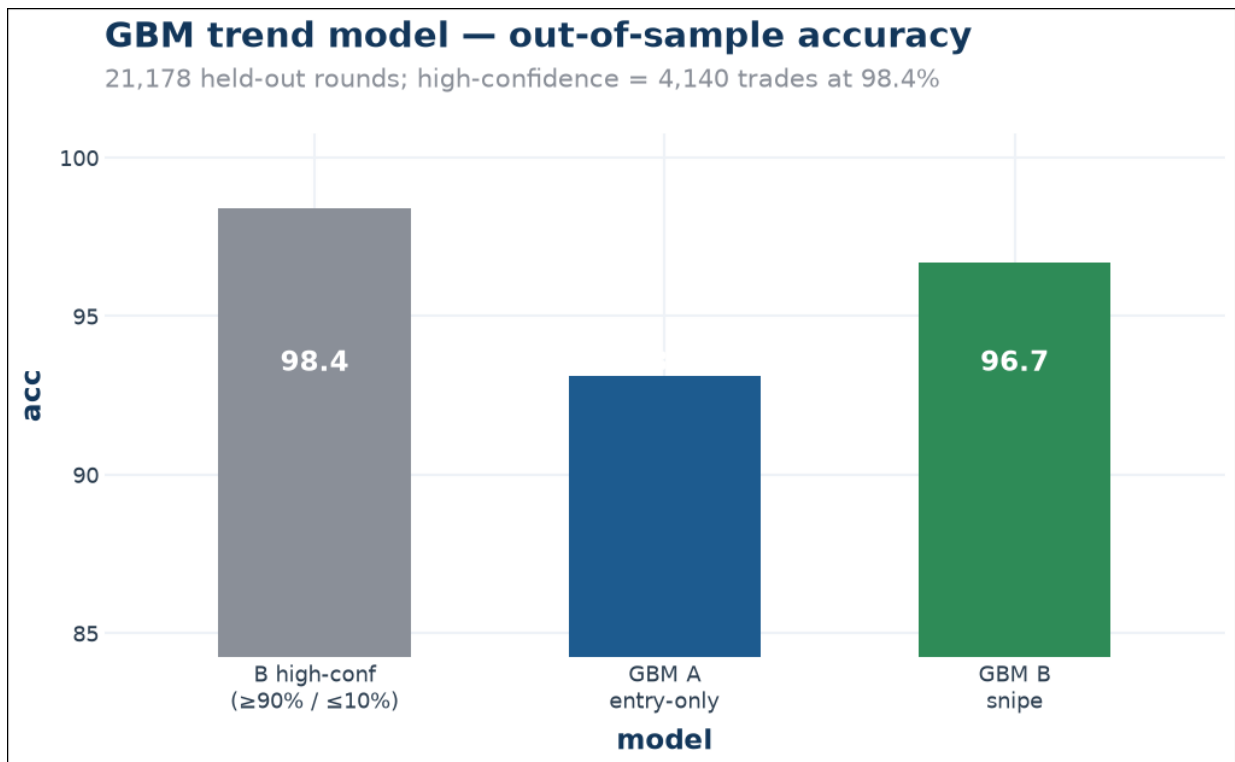
Every row in the dataset is structured the same way:

- **entryPoint**: the snapshot at close-minus-3-minutes - implied price, opening implied, deltas, spread, volume rate, open-interest change, range, hour, and the last three outcomes of the series.
- **previewCandles**: the last eight one-minute candles before the decision point.
- **trendPath**: the candle path after decision to close - the suspected trend the model learns to read.
- **resolution** and **fills**: where it landed, plus the account's real trades on that round.

Then we audited the data for leaks, because a model that reads the answer before it answers isn't a model, it's a cheat sheet. The audit passed: no future candles, no same-closeTime parallel-strike outcomes leaking into history features (48,109 excluded), no suspicious train/test correlations, and a strictly chronological split.

Out of that dataset we trained a gradient-boosted model - 200 trees, depth three, built in pure JavaScript with no external ML dependencies, so the whole thing runs on one machine. On rounds it had never seen:

Model	What it sees	Accuracy (out-of-sample)
Entry-only	the snapshot before you enter	93.1%
Snipe	plus the first two minutes of the trend	96.7%
Snipe, high-confidence	its strongest calls only	98.4% (4,140 trades)



The model's out-of-sample accuracy, strongest to strongest

We also built the honest evaluation loop: a dashboard that plays every round back, shows the model's reasoning, and scores it against reality. The dashboard only evaluates rounds the model never trained on, so what you see is what the model would actually do - not a backtest with the answers visible.

4. How it works now: three layers and the exact rules

Watch a good trader for ten minutes and you'll notice they don't compute, they observe - the book, the candles, the way the market is moving - and they ask one question: does this look like the setups that win? Kalshiman is three people in one.

A statistician. The GBM model. It has read 105,000 finished rounds and knows, to the percentage point, how often a 92-cent gold side actually settles. When it says 98%, it's right 98.4% of the time. It is calibrated, and it is never surprised.

A quiet observer. A small language model (Qwen3-1.7B, running locally via Ollama) reads each round the way a person would - the book, the path, the history - and gives a one-line opinion: yes, no, or pass. It is often wrong on its own, which is exactly why it is never in charge. Its opinions are written down and scored later, like a trader's journal.

A disciplined hand. It only acts when the statistician is confident and the observer agrees. Once in, it does the hard part: it lets a dip breathe, and if it does leave, it leaves at the stop level - never at whatever panicked price the market printed while it wasn't looking.

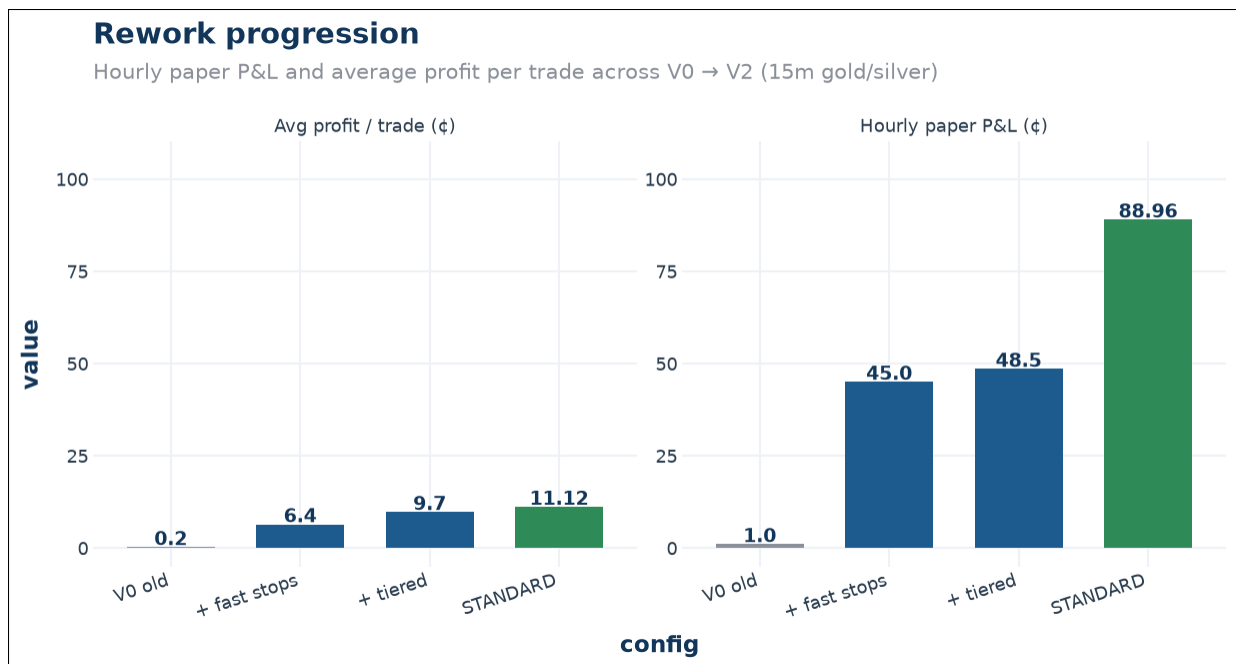
Step	What the code actually does
Scan	every 5s: gold15-late, silver15-late, universal snipe (0.5-2.5m to close)
Score	researcher pulls 1m candles; logistic + GBM + GBM-snipe probabilities
Gate	price band 80-98c, timing, EV, liquidity - then the GBM bar: $\geq 85\%$ or $\leq 15\%$ and agreement
Vetoes	gbm-low-conf · gbm-disagree · gbm-pending · gbm-warmup · gbm-timing (<4m to close below 95c)
Enter	paper trade, \$1
Manage	every tick + a 1.5s fast stop loop (paper AND live)
Win	bank at 99-99.5c or hold to resolution
Loss	tiered stop 15c ($\geq 90c$ entries) / 8c; 3-minute grace; limit-stop fill at the stop level
Final 2 min	GBM-confirmed trades hold to resolution - no stop (the market converges 96-99% by then)
After	written post-mortem: result, heldPnl, verdict - feeds retraining

Every rule in that table exists because a specific losing pattern taught it to us. The three-minute grace exists because nine of fifteen stops used to recover and win. The stop-level fill exists because a 15-cent stop once booked a 50-cent loss. The entry-timing guard exists because a trade entered 3.6 minutes before the bell has no time to survive a dip and recover. This is a system that keeps its receipts.

5. The receipts

Here is the rework, hour by hour, on paper. Each row is the change that made the next row possible.

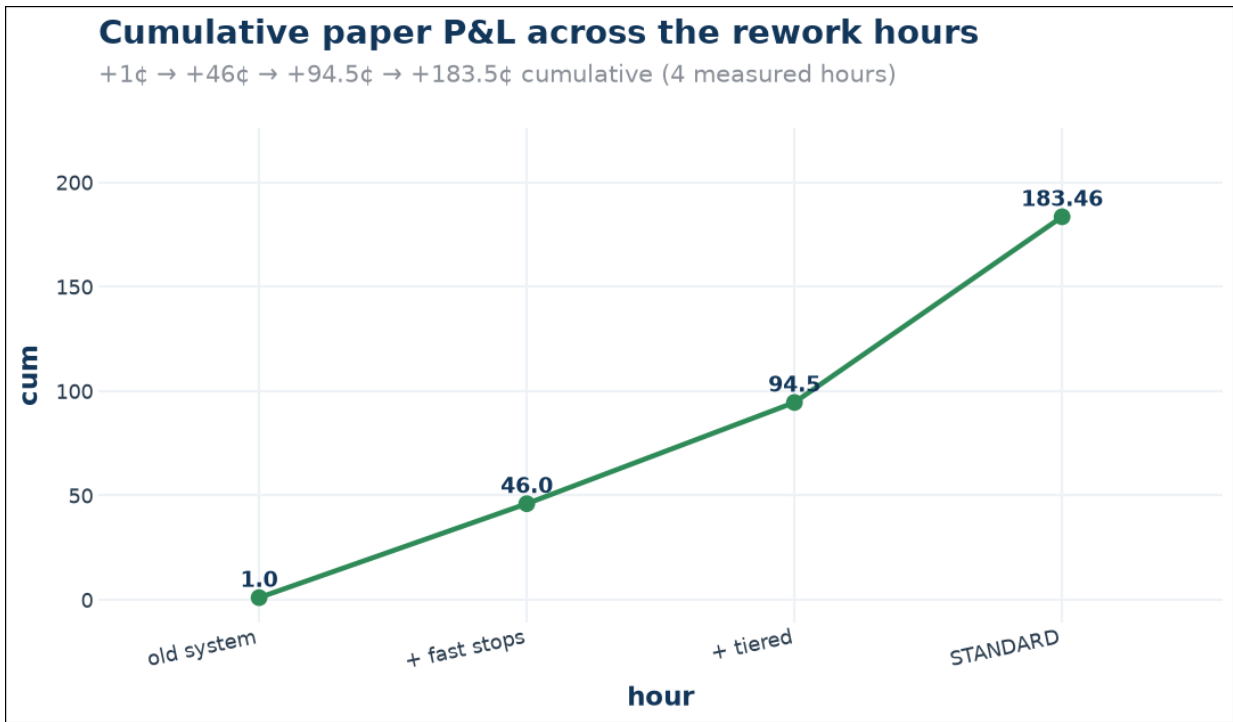
What changed	Trades	Result	Avg/trade
The old system	5	+1c, one loss	+0.2c
Paper stops that actually work	7	+45c, one loss	+6.4c
Stops that respect the entry	5	+48.5c, no losses	+9.7c
Stops that wait out noise	8	+88.96c, no losses	+11.12c



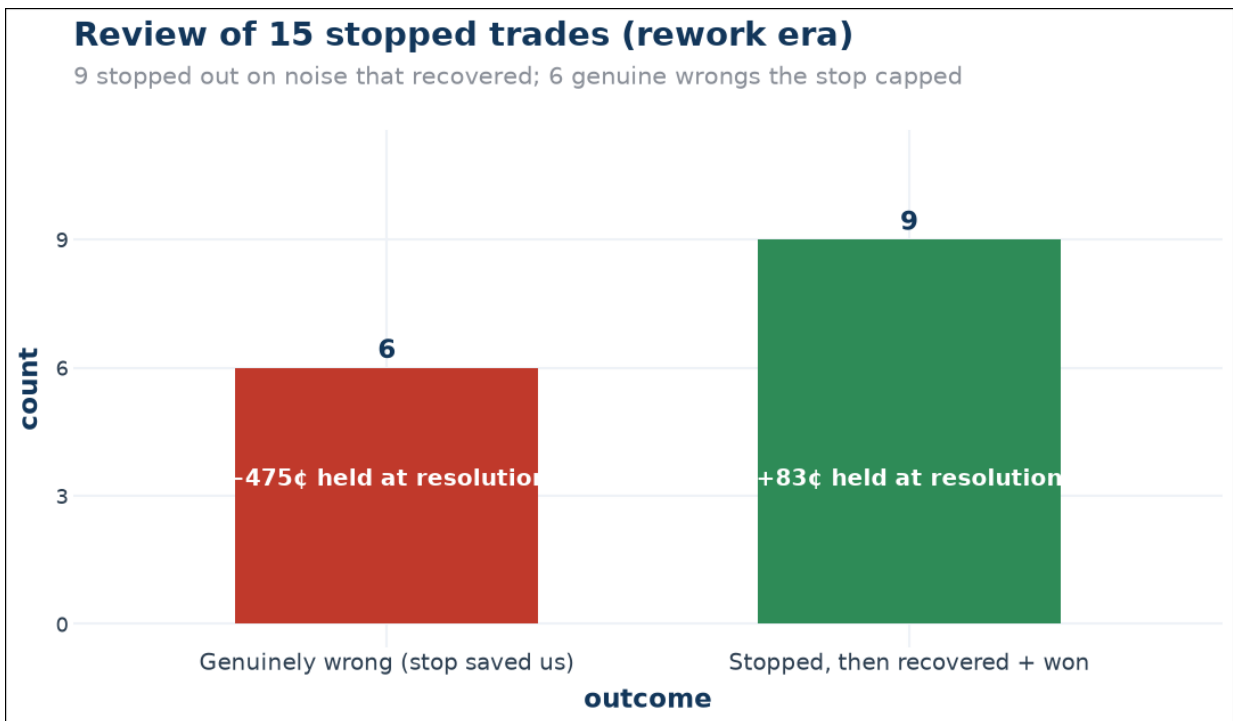
The same story as a chart: money per hour, and money per trade

The standard hour, trade by trade: gold NO at 90c banked 6.74c. Silver YES at 81c held to the bell for 23c. Gold NO at 93c banked 2.74c. Silver NO at 93c held for 8c. Gold YES at 87c banked 4.74c. Silver YES at 86c held for 16c. Gold NO at 93c banked 4.74c. Silver NO at 81c held for 23c. No heroics. The bot let the market do the work and got out of its way.

Through the first evening of the standard config: 21 trades, 20 wins, 1 loss - and the one loss was capped at 6.74c at the stop level. Live prediction accuracy: 89.5% model, 99% market. And the account what-if from real fills: trades that agreed with the model won 92.5%; trades that went against it won 6.4%.



Cumulative paper P&L; across those hours



Why the stops changed: nine recovered and won, six were genuine wrongs

6. Risk, and the stop policy

A stop is a promise you make to yourself before the trade: if this goes wrong, it costs me this much and not a cent more. The old bot broke that promise in both directions - it fired on noise, and when a real break happened it filled at whatever the market panicked to. The new policy fixes both.

The evidence base is the rework's fifteen stops: nine fired on transient dips that recovered and won (the flat stop cost us 188c of forgone wins), and six were genuine wrongs where the stop saved 344c. Net, stopping still saved money - the problem was never stopping, it was stopping on noise and filling at crash prices.

Control	What it does
Tiered stops	entry $\geq 90c$ gets 15c of room; below that, 8c - high prices mean the dip is almost always noise
Recovery grace	a dip has 3 minutes to prove it's real; if the market comes back, the winner stays
Limit-stop fills	the exit is marked at the stop level - a crash price is never booked
Fast checks	every 1.5 seconds, paper and live
Entry timing	no mid-price entry ($< 95c$) with under 4 minutes to close - a dip needs room to recover
Final-minute hold	GBM-confirmed entries in the last 2 minutes ride to resolution
Hard bans	no martingale, no revenge trading, no leverage, no both-sides arbs
Real money	off until the paper record earns it

The two newest rules came from the same loss. A silver trade entered 3.6 minutes before the bell dipped and was stopped 52 seconds before close - the closing-soon rule fired, and no grace length could help. The fixes: don't take that entry at all (timing guard), and for the trades you do take in the final minutes, hold them to the bell (final-minute hold) because the market has already converged. The loss that taught us both: capped at 6.74c, exactly at the stop level.

7. The economics - one bot, a fleet, a thousand bots

The unit of edge is a percent, not a cent. Buy at 90 cents, it settles at 100: +11.1% on what you deployed. The cents in our reports are the per-contract unit; the dollars scale with position size - and position size scales with the balance you're willing to back it with.

Why growth accelerates: the return is a percentage of your stake, and your stake is a percentage of your balance. At \$200 balance and a \$50 entry, ten perfect trades are +\$55.60 - the resolution stops making cents and starts making dollars. The ladder:

Balance	Stake (25%)	One win (+11%)	One stop
\$5	\$1.25	+\$0.14	-\$0.11
\$30	\$7.50	+\$0.83	-\$0.67
\$100	\$25	+\$2.78	-\$2.22
\$200	\$50	+\$5.56	-\$4.44
\$500	\$125	+\$13.89	-\$11.11

Now the network math. Assume the current strategy, a 95.2% win rate (one miss in every 21 trades), and every bot eventually trading \$20 stakes from a \$200 balance. A win pays \$2.22; a capped stop costs \$1.78; a full final-minute loss costs \$20. Per trade, the expected net is about +\$2.03 with capped stops, or +\$1.16 if the rare miss is a full loss.

Bots	Trades/day each	Per-bot / month	Network / month
1	30	\$1,829	\$1,829
1	100	\$6,095	\$6,095
10	100	\$6,095	\$60,950
100	100	\$6,095	\$609,500
1000	100	\$6,095	\$6,095,000

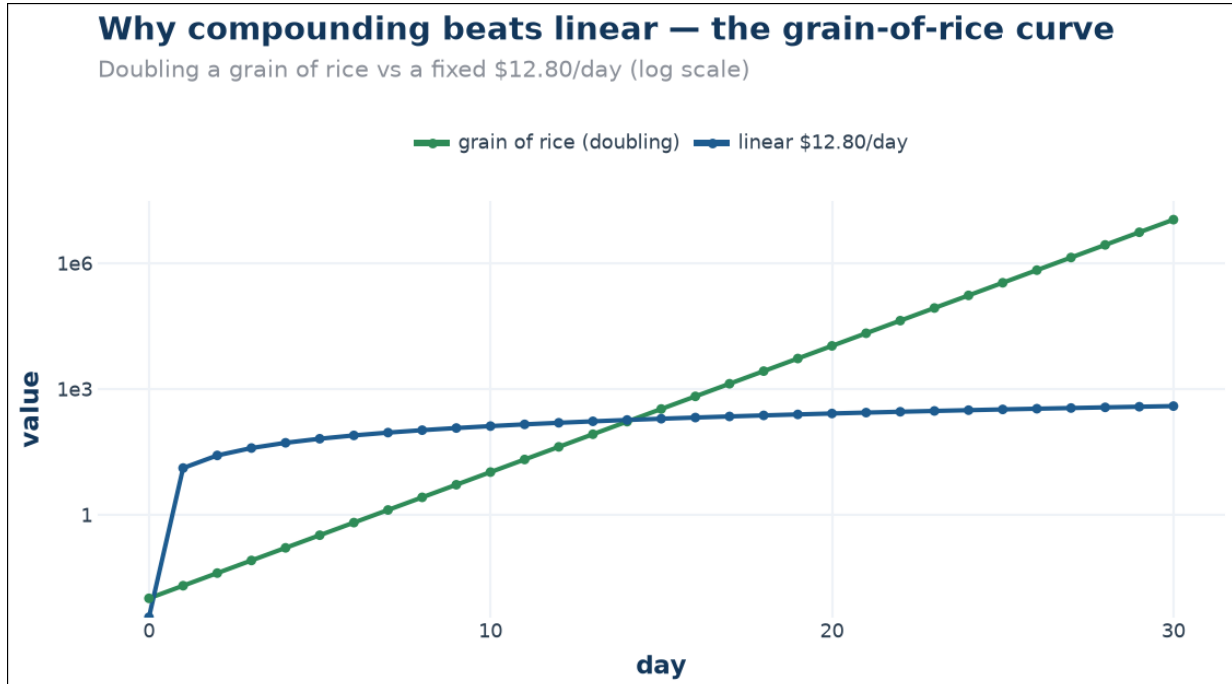
What that means for real people: a family of four in the US can live on about \$1,800 a month - one bot at 30 trades a day covers that. A college student covering food and fun needs about 10 trades a day. A group of friends running ten bots together is making hobby money on autopilot. A determined individual running a hundred is looking at quitting their job. A thousand bots is the extreme goal - and we say it plainly: this is the direction, not the promise. Two things cap it before the math does:

- **Liquidity.** A 15-minute round has maybe \$50-100k of depth. A thousand bots can't all trade gold at once
- which is exactly why the fleet must spread across hundreds of markets, not pile into one.

- **The loss tail.** At \$20 stakes, a full final-minute loss costs \$20, and one of those per 21 trades drags the per-trade net from \$2.03 to \$1.16. The win rate is the second-most-important number in the whole system, after the stop.



Flat per-trade scenario math - before position sizing



Why compounding beats linear - the grain-of-rice curve

8. The pool - how a fleet works

One bot can watch a handful of markets every night. Ten bots can watch all of them, and each one gets to be lazier because the others cover the rest. The pool is the nervous system that lets workers share what they find instead of each learning everything alone.

The players. The main node owns the strategy configs and is the only one that pushes verified strategies down. Workers run the same engine on their own slice. A small Cloudflare Worker in front of a KV store is the shared inbox: signals, ledgers, strategy versions.

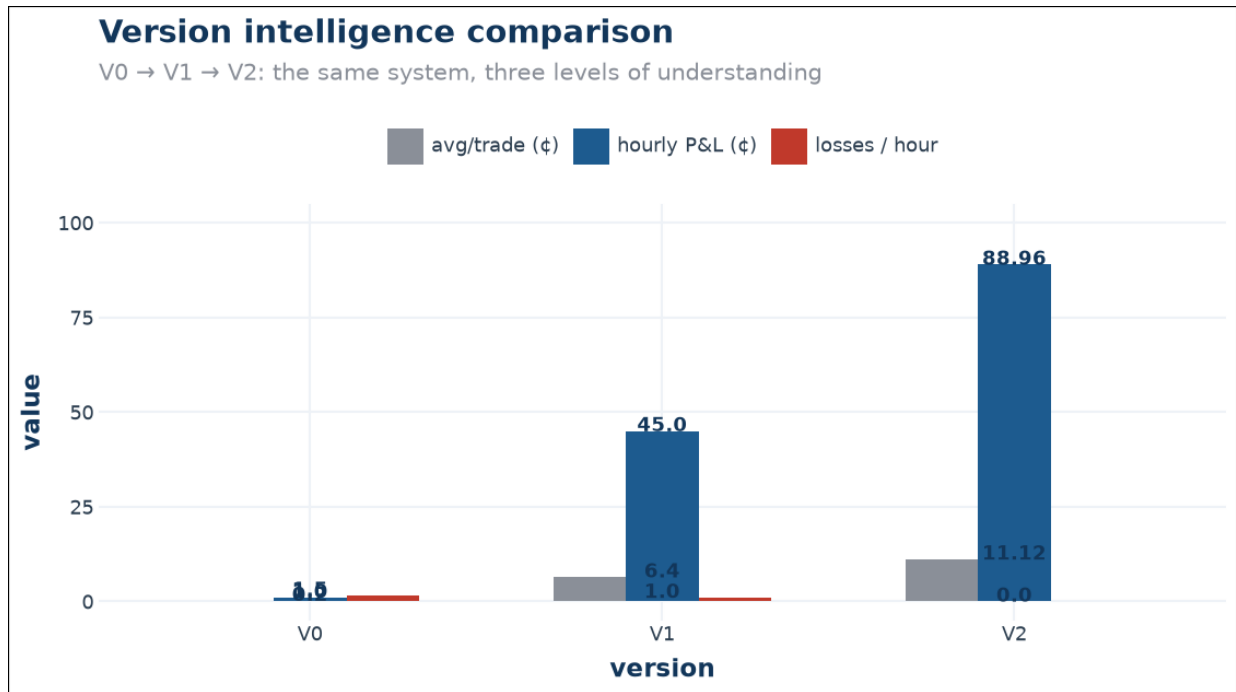
The credits economy. Publishing a good signal costs a few credits in escrow. Another worker who verifies it gets paid for the verification, and the original poster is refunded when the signal proves actionable - so posting good information pays for itself, and posting garbage costs. A single piece of information can only be refunded once. Workers cap at 200 credits; anything above 100 flows into the pool's float, which is used to verify incoming information, so the network stays self-balancing.

The guardrails. Any node can publish at most one post per 10 minutes; any node can verify at any time; only the main node pushes strategy versions. Workers specialize naturally - a node in another timezone covers the overnight markets, a commodity-only node runs just those lanes - and because everyone shares for a flat, small rate, the fleet converges toward the same knowledge over time. That's the daisy chain: one worker's discovery becomes everyone's.

Current status, honestly. The pool is disabled during the paper test. The live worker burned its daily KV quota through sync fan-out, not trading - the /feed endpoint did a KV list and a read fan-out on every call, and one idle node polling every 60 seconds through three overlapping paths used up the free tier in under a day. Fixed client-side (5-minute polls, dedupe); a worker cache is staged and needs deploying from the account that owns the pool. The roadmap: deploy the cache, re-enable, run two nodes on paper side by side, then scale positions within each market's liquidity - not beyond it.

9. Where we're going

The bot didn't get smarter by adding parameters. It got smarter by learning from its own outcomes, refusing weak setups, and finally understanding the difference between noise and signal. That's not magic - it's a trader who started keeping a journal.



V0 to V2: the same system, three levels of understanding

What's next is boring in the best way: more markets with trained models (16 series are in the dataset; two trade today), a slightly earlier entry window on the setups the model is sure of, day after day of net-positive paper hours, then the sizing ladder - \$1 paper stakes today, \$5, \$20, \$50 as the record earns it. Then the fleet: two nodes on paper side by side, sharing signals, then a real network of workers in different markets and timezones.

One perfect hour isn't a track record. We know that. But for the first time since we started, the bot isn't the reason we're not there yet.

10. Straight answers

What is the actual edge?

Ultra-short markets converge: a side implied at 90%+ settles that way ~96-99% of the time. We enter late, with a confident model and an observer's agreement, ride winners through noise, and cap the rare wrong call at a fixed stop level.

Why did the old system fail?

It was a static formula with no sense of context, stops that gaped 17-65c on paper, and no journal. The rework fixed all three, and every fix maps to a measured loss.

What is the LLM for?

It's the observer - it reads each round like a trader would and gives a one-line opinion. Advisory only, logged, scored, never in charge.

Why a three-minute grace on stops?

Because nine of fifteen stops in the old system fired on dips that recovered and won. A dip gets time to prove it's real; a break that stays still closes at the stop level.

What are realistic returns?

Paper standard hour: 8W/0L, +88.96c, 11.12c average, \$0 drawdown. At \$20 stakes with a 95.2% win rate, one bot projects ~\$60-200 a day depending on how many trades the market offers.

When can this go live?

After day after day of net-positive paper hours, a passing leakage audit, and confirmation that stops fill at the stop level. Then the sizing ladder begins at \$1.

What are the biggest risks?

A real reversal on a high-price entry (capped at the stop), a regime change that breaks the pattern until the journal catches it, Kalshi fee or availability changes, liquidity limits at scale - and one perfect hour not being a track record.

Does it use leverage?

No. No martingale, no revenge trading, no leverage, no both-sides. Real money is off until the paper record earns it.

Why run it at all?

A few watts for a self-correcting system with a quantified edge and capped downside - and the first worker in a fleet that makes quiet days into free gains.

Disclaimer: Past performance is not a guarantee of future results. Trading involves real risk. Kalshi is not available in all states. Nothing in this document is financial advice. All figures are read live from the system state and Kalshi settlement records on 2026-08-20 04:36 UTC.

Companion markdown: docs/00-thesis.md, docs/06-performance.md, docs/13-versions.md, docs/14-reasoning.md, docs/15-returns.md, docs/18-pool.md, ARCHITECTURE.md. Charts: docs/charts/*.png (plotnine).